

实验报告：构建 Karpathy 风格长上下文知识库

马汝坡 杭州电子科技大学 大二电子信息工程 学号：24200516

2026 年 4 月

1 1. 架构对比分析（重点）

1.1 1.1 传统 RAG 与 Karpathy 方案对比

维度	传统 RAG	Karpathy 长上下文方案（本项目）
数据流程	分块、向量化、检索 Top-K、生成	raw 资料输入、LLM 整库阅读、结构化 Markdown 输出
召回机制	依赖检索结果，可能漏召回	整库进入上下文，召回更完整
跨文档推理	受检索片段限制	单次上下文内可直接跨页综合
系统复杂度	需要向量库和检索组件	文件系统 + LLM API 即可
适用规模	中大规模知识库	个人/课程级中小规模知识库

1.2 1.2 本项目设计决策

1. 使用 Markdown 作为唯一存储：便于 Obsidian 展示与人工维护。
2. 不引入向量数据库：遵循课程“极简架构”要求。
3. 同时保留两条编译路径：`build_kb.py` 用于稳定样例，`compile_raw_llm.py` 用于自动编译原始资料。
4. 问答采用整库上下文输入：保持“非检索、长上下文”原则。

1.3 1.3 长上下文方案局限性

- Token 成本随知识库规模增加而明显上升。
- 输入变长导致响应时延上升。
- 当知识库规模超过上下文窗口时，需要分层编译与分主题组织。

2 2. Prompt 工程细节

2.1 2.1 Compilation Prompt 结构

在 `src/compile_raw_llm.py` 中，Prompt 规定模型输出严格 JSON，核心字段为：`site_title`，`pages[].title`，`summary`，`highlights`，`equations`，`questions`，`tags`，`related`。

关键约束：

1. pages 数量至少 10。
2. related 必须引用 pages 中存在的标题。
3. 输出语言为中文，主题聚焦电磁场与电磁波课程。

2.2 2.2 Prompt 迭代优化

- 初版问题：输出混入解释文本，JSON 解析失败。
改进：增加“只输出 JSON”并启用 JSON 模式。
- 第二版问题：页面数量不足。
改进：硬约束 $\text{pages} \geq 10$ 。
- 第三版问题：related 指向不存在页面。
改进：后处理校验并过滤无效引用。

3 3. 测试案例

3.1 案例一：静电场与高斯定律关系

问题：高斯定律和静电场有什么关系？

跨文档页面：矢量场、坐标系与库仑定律；高斯定律与电通量；麦克斯韦方程组与统一图景。

3.2 案例二：位移电流的作用

问题：位移电流为什么能把静电理论扩展到电磁波理论？

跨文档页面：时变电磁场与电磁感应；麦克斯韦方程组与统一图景；均匀平面波与传播特性。

3.3 案例三：传输线与平面波联系

问题：传输线分析和均匀平面波分析在工程上有什么联系？

跨文档页面：均匀平面波与传播特性；传输线、驻波与波导；辐射、天线与远场。

3.4 编译前后截图说明

提交版 PDF 需包含：raw 目录截图、wiki 目录截图、问答终端截图、Obsidian 图谱截图。

4 4. 性能评估

4.1 4.1 知识库规模（实测）

- Markdown 文件数：11（索引 1 + 章节 10）
- 总字符数：6297
- 中文字符数：3018
- Token 估算：约 3704（按 chars/1.7 粗估）

4.2 4.2 编译耗时与问答响应时间（实测）

- `python src/build_kb.py`: 约 0.095 s
- `python src/qa.py " 高斯定律和静电场有什么关系"`: 约 0.222 s（本地兜底模式）

4.3 4.3 API 调用成本估算

采用统一估算公式：

$$\text{cost} = \frac{T_{in}}{10^6} P_{in} + \frac{T_{out}}{10^6} P_{out}$$

示例：若一次问答输入 4000 token、输出 800 token，按输入 2 元/百万、输出 8 元/百万估算，单次约 0.0144 元。

5 5. 反思与展望

5.1 5.1 适用与不适用场景

适用：个人学习知识库、课程项目、文档规模可被长上下文覆盖的场景。

不适用：百万字以上知识库、高并发低延迟在线服务。

5.2 5.2 面向 100 万字规模的改进方向

1. 分层编译：先分主题子库，再维护总索引。
2. 增量更新：仅重编译受影响页面。
3. 轻量索引路由：关键词路由到子库后再长上下文回答。
4. 成本控制：批处理编译和结果缓存。

6 结论

本项目已完成课程要求中的核心闭环：Markdown 知识库组织、长上下文问答、图谱导出、自动编译入口。后续重点优化方向为：增量编译与真实 API 场景下的系统化评测。